

Lane & Obstacle Detection

1. README / Code Execution

To evaluate the lane and obstacle detection algorithm, run the `run_gold.py` script from the command line. Please keep in mind that:

- **Prerequisites:** The implementation requires Python 3 and the `opencv-python`, `numpy`, and `ultralytics` (YOLO) packages. It is highly recommended to execute the code within a Python Virtual Environment (`venv`) to prevent dependency conflicts.
- **Execution & Debug Mode:** The script must be launched by passing the target dataset path as an argument (e.g., `python run_gold.py "PandaSetSensorData/archive/044/Camera/front_camera/*.jpg"`). It is possible to append the `debug` keyword at the end of the command to activate a 2x2 grid showing the internal algorithm steps (Binarization and Histogram). *Disclaimer: this feature is provided strictly as an extra diagnostic tool and is not intended as the default execution mode.*
- **User Controls:** During the continuous video-like execution, the program can be safely terminated at any time by pressing the `q` or `ESC` keys on the keyboard, or by clicking the standard close button (`X`) on the window. Furthermore, **only in debug mode**, the user can pause and resume the execution by pressing the `SPACE` bar.

2. GOLD Algorithm Implementation

This section details the logical and mathematical architecture of the software, following the fundamental steps of the GOLD (Generic Obstacle and Lane Detection) algorithm.

2.1 Bird's Eye View (BEV)

The perspective transformation removes the camera's foreshortening effect to obtain a top-down view where lane delimiters appear parallel.

- **Projection Model:** Using the provided camera parameters—specifically the focal length ($f_x = 1970$, $f_y = 1970$), the principal point ($c_x = 970$, $c_y = 483$), and the camera height ($h = 1.66$ m), a set of points in the 3D world (with a constant Z on a flat road plane) was defined and projected onto the 2D image plane.
- **IPM (Inverse Perspective Mapping):** Through the `cv2.getPerspectiveTransform` function, the homography matrix M was calculated to map the frontal view trapezoid into an output rectangle (the **BEV**). The resolution was set to 20 pixels per meter

(`px_m = 50`), covering a region of interest ranging from 5.5 to 25 meters ahead of the vehicle.

2.2 Pre-processing: Greyscale and Blur

Before feature extraction, the BEV image undergoes two essential cleaning steps:

- **Grayscale Conversion:** This step reduces computational complexity while retaining the luminance information necessary to distinguish the bright road paint from the darker asphalt.
- **Gaussian Blur:** A 5 x 5 kernel blur is applied to mitigate high-frequency noise and small imperfections in the road texture, smoothing the image in preparation for spatial filtering.

2.3 Lane Enhancement

To highlight lane lines and suppress distracting elements (such as shadows or dark vehicles), a separable 2D filter was implemented.

- **Spatial Kernels:** Two specific kernels were defined: $g(x)$ (acting as the second derivative of a Gaussian, or *Mexican Hat*) and $g(y)$ (a standard Gaussian profile).
- **Filter Effect:** The $g(x)$ kernel acts as a *Ridge Detector*: it responds positively to narrow, bright peaks (the white stripes) while zeroing out areas of constant brightness. The resulting negative values, which correspond to objects darker than the surrounding asphalt, are truncated to zero to effectively "clean" the semantic scene.

2.4 Iterative Binarization

The enhanced image is converted into a binary format (black and white) using an adaptive thresholding algorithm.

- **Algorithm:** Starting with an initial threshold set at the midpoint between the minimum and maximum grayscale values, the algorithm recursively divides the image pixels into two regions (A and B). The threshold is then updated as the average of the mean pixel values of these two regions. The iteration stops when the threshold variation drops below 0.5. This dynamic approach guarantees an optimal extraction of the lane stripes regardless of global lighting conditions.

2.5 Histogram Calculation and Peak Detection

The final phase locates the exact horizontal position of the lanes within the binarized image.

- **Histogram Construction:** The binary image is projected onto the X-axis by summing the white pixel values along each vertical column.
- **Peak Search:** The horizontal range is split into two halves (left and right). In each half, the maximum value is identified (`np.argmax`). To prevent false positives caused by residual noise, a peak is validated only if its amplitude exceeds 10% of the total BEV height (acting as a confidence threshold). The X coordinates of these validated peaks identify the lane positions, which are then drawn as green vertical lines on the final output.

3. More Implemented Features

This section describes the implementation of other advanced features.

3.1 Continuous vs. Dashed Lane Discrimination

The algorithm distinguishes between continuous and dashed lane delimiters by analyzing the vertical distribution of white pixels within the detected lane column in the binary BEV image.

- **Metric Calculation:** A "vertical fill ratio" is computed by checking the percentage of rows in the lane column that contain at least one white pixel.
- **Classification:** If the ratio is high (e.g., > 55%), the line is classified as continuous. A lower ratio indicates a dashed line. This allows the system to change the visualization style accordingly (solid vs. dashed green lines).

3.2 Visualization on the Original Image

The detected lanes are projected back from the Bird's Eye View to the perspective of the original camera.

- **Inverse Mapping:** Using the inverse of the homography matrix (M_{inv}), the vertical coordinates of the lanes identified in the BEV are re-mapped into the original image space.

3.3 Obstacle Detection and Distance Estimation

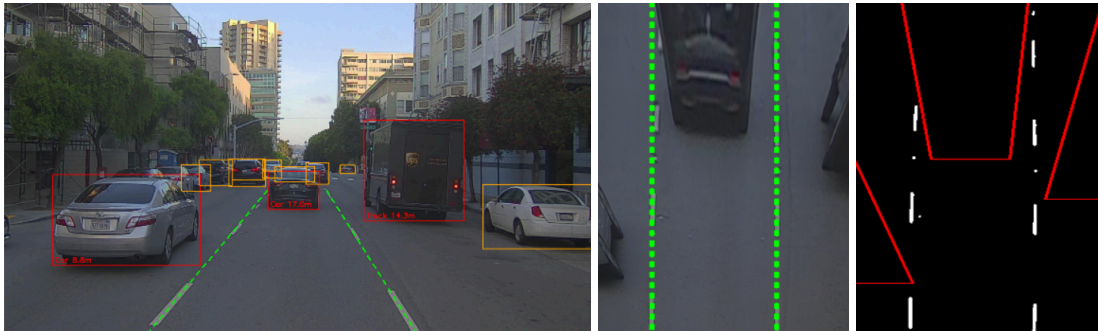
The system integrates a deep learning-based object detector to identify potential hazards in the vehicle's path.

- **YOLO Integration:** The YOLOv8 (You Only Look Once) model is utilized to detect objects such as cars, trucks, and persons in the original frame.
- **Distance Mapping:** The bottom center of the detected bounding box (representing the point where the tires touch the road) is projected into the BEV space using the M matrix.
- **Metric Estimation:** Based on the y position in the BEV and the calibrated pixel-per-meter scale (px_m), the algorithm calculates the approximate longitudinal distance in meters from the camera sensor to the obstacle.
- **In-BEV Filtering:** The algorithm checks if the projected vehicle coordinates fall within the BEV boundaries: vehicles inside the BEV area are highlighted with a red bounding box and their relative distance/class labels; vehicles outside this area are simply marked with an orange bounding box without labels.

3.4 ENHANCEMENT: Obstacle Masking (not requested)

A significant improvement was implemented to prevent false positives in lane detection caused by the high contrast of vehicle edges. We utilized the YOLO detection results (the Bounding Boxes) to perform "Obstacle Masking". For every detected vehicle, a polygonal

mask is generated based on its bounding box and projected into the BEV-binarized image; before the histogram calculation, these areas are filled with black pixels, ensuring that the **histogram peaks correspond only to actual road markings** and not to the vertical edges of surrounding traffic.

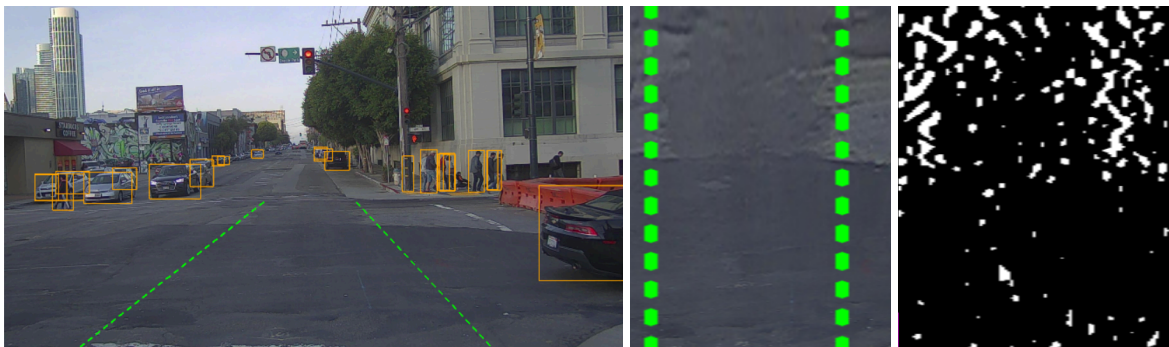


(The red bounding boxes in the binarized-BEV image **do not** contribute to the white pixel number calculation for the histogram, they're just shown for debug purposes.)

4. Common Issues

4.1. Flat Images

Images characterized by flat lighting and colors can result in highly noisy binarized outputs. Even with the application of Gaussian blurring and spatial lane enhancement, this can occasionally lead to erroneous lane detections.



4.2. Lines of parked cars

In some cases, a continuous line of parked cars along the roadside can trigger a false lane detection if the YOLO bounding box masking partially fails to cover the entire vehicle, or if the vehicles' bottom edges align perfectly, generating a peak in the histogram.



4.3. Other

Other high-contrast features on the ground (such as zebra crossings, clear linear marks/cracks on the asphalt, or physical obstacles that are not recognized by the YOLO model) can introduce vertical artifacts in the BEV, leading to a misinterpret of these elements as valid lane delimiters.